



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

Lập trình Hướng Đối tượng Object Oriented Programming

\$10 Ngoại lệ - Exception





Nội dung

- Phát sinh và bày ngoại lệ
- Đối tượng ngoại lệ
- Tạo riêng ngoại lệ





ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

1. Ngoại lệ - Exception



Ngoại lệ

- Định nghĩa: Ngoại lệ là một sự kiện xảy ra trong quá trình thực thi chương trình, phá vỡ luồng bình thường của chương trình

Ngoại lệ ~ Exceptional event

- Ví dụ

- `int i = 4/0;`

Exception

Ngoại lệ

- Ngoại lệ là một lỗi đặc biệt
- Xảy ra trong thời điểm chạy chương trình (runtime)
- Khi xảy ra một ngoại lệ, nếu không xử lý thì chương trình kết thúc ngay và trả lại quyền điều khiển cho hệ điều hành.
 - Kết thúc bất thường
 - Kết quả thực thi không mong muốn

Xử lý lỗi truyền thống

- Viết mã xử lý tại nơi phát sinh ra lỗi =>
 - Chương trình trở nên rối
 - Không phải lúc nào cũng đầy đủ thông tin để xử lý
 - Không nhất thiết phải xử lý
- Truyền trạng thái lên mức trên
 - Thông qua tham số, giá trị trả lại hoặc biến toàn cục
 - Dễ nhầm
 - Khó hiểu

```
if( func()> 0 )  
    // Return value positive => o.k.  
else  
    // Treat errors
```

Xử lý lỗi truyền thống

Nhược điểm

- Khó kiểm soát được hết các trường hợp
 - Lỗi số học, lỗi bộ nhớ,...
- Lập trình viên thường quên/bỏ sót không xử lý lỗi
 - Bản chất con người
 - Thiếu kinh nghiệm, cố tình bỏ qua



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

2. Bẫy và xử lý ngoại lệ

try catch

Mục đích của xử lý ngoại lệ

- Tách biệt khối lệnh xử lý (có thể phát sinh ngoại lệ) và khối lệnh xử lý ngoại lệ
- Giúp chương trình đáng tin cậy hơn, tránh kết thúc bất thường

```
try
{
    cmd.ExecuteNonQuery();
    ErrorsManager.SetError(ErrorIDs.KhongCoLoi);
}
catch (Exception ex)
{
    ErrorsManager.SetError(ErrorIDs.SQLThatBai,
        database.DbName, "ten_store");
}
```

```
.....
IF B IS ZERO GO TO ERROR
C = A/B
PRINT C
GO TO EXIT
```

```
ERROR:
DISPLAY "DIVISION BY
ZERO"
```

```
EXIT:
END
```

} Khối xử lý lỗi

Mục đích của xử lý ngoại lệ - Phân tách code

Lập trình truyền thống => nhiều đoạn mã để xl lỗi

```
errorCodeType readFile {
    initialize errorCode = 0;
    open the file;
    if (theFileIsOpen) {
        determine the length of the file;
        if (gotTheFileLength) {
            allocate that much memory;
            if (gotEnoughMemory) {
                read the file into memory;
                if (readFailed) {
                    errorCode = -1;
                }
            }
        }
    }
}
```

```
        else {
            errorCode = -2;
        }
        else {
            errorCode = -3;
        }
    }
    close the file;
    if (theFileDidntClose && errorCode == 0) {
        errorCode = -4;
    } else {
        errorCode = errorCode and -4;
    }
} else {
    errorCode = -5;
}
return errorCode;
```

Mục đích của xử lý ngoại lệ - Phân tách code

Cơ chế bắt ngoại lệ

```
readFile {  
    try {  
        open    the    file;  
        determine its size;  
        allocate that much memory;  
        read    the    file    into  
        memory; close the file;  
    } catch (fileOpenFailed)  
        { doSomething;  
    } catch (sizeDeterminationFailed)  
        { doSomething;  
    } catch  
    (memoryAllocationFailed)  
    {  
        doSomething;  
    } catch (readFailed) {  
        doSomething;  
    } catch (fileCloseFailed)  
        { doSomething;  
    }  
}
```

Cơ chế ngoại lệ cho phép tập trung viết mã cho luồng chính và xử lý trường hợp bất thường ở nơi khác

Mục đích của xử lý ngoại lệ -

- Tránh việc lan truyền mã lỗi trong chuỗi các lời gọi thủ tục
- VD

```
method1 {  
    call method2;  
}  
  
method2 {  
    call method3;  
}  
  
method3 {  
    call readFile;  
}
```

Mục đích của xử lý ngoại lệ - Phương pháp lan truyền lỗi truyền thống

```
method1 {  
    errorCodeType error;  
    error = call method2;  
    if (error)  
        doErrorProcessing;  
    else  
        proceed;  
}
```

```
errorCodeType method2 {  
    errorCodeType error;  
    error = call  
    method3; if (error)  
        return error;  
    else  
        proceed;  
}
```

```
errorCodeType method3 {  
    errorCodeType error;  
    error = call  
    readFile; if (error)  
        return error;  
    else  
        proceed;  
}
```

*Bắt buộc method2 và method3 truyền mã lỗi trả về bởi **readFile** cho đến khi mã lỗi tới được method1 - phương thức duy nhất quan tâm đến việc xử lý lỗi.*

Mục đích của xử lý ngoại lệ - Tránh việc lan truyền mã lỗi

- Phương thức có thể phát sinh ngoại lệ => cho Phương thức gọi nó bấy
chỉ phương thức nào quan tâm xl lỗi mới cần bấy lỗi

```
method1 {  
    try {  
        call method2;  
    } catch (exception e) {  
        doErrorProcessing;  
    }  
}  
  
method2 throws exception {  
    call method3;  
}  
  
method3 throws exception {  
    call readFile;  
}
```

Mục đích của xử lý ngoại lệ

Nếu không xử lý ngoại lệ

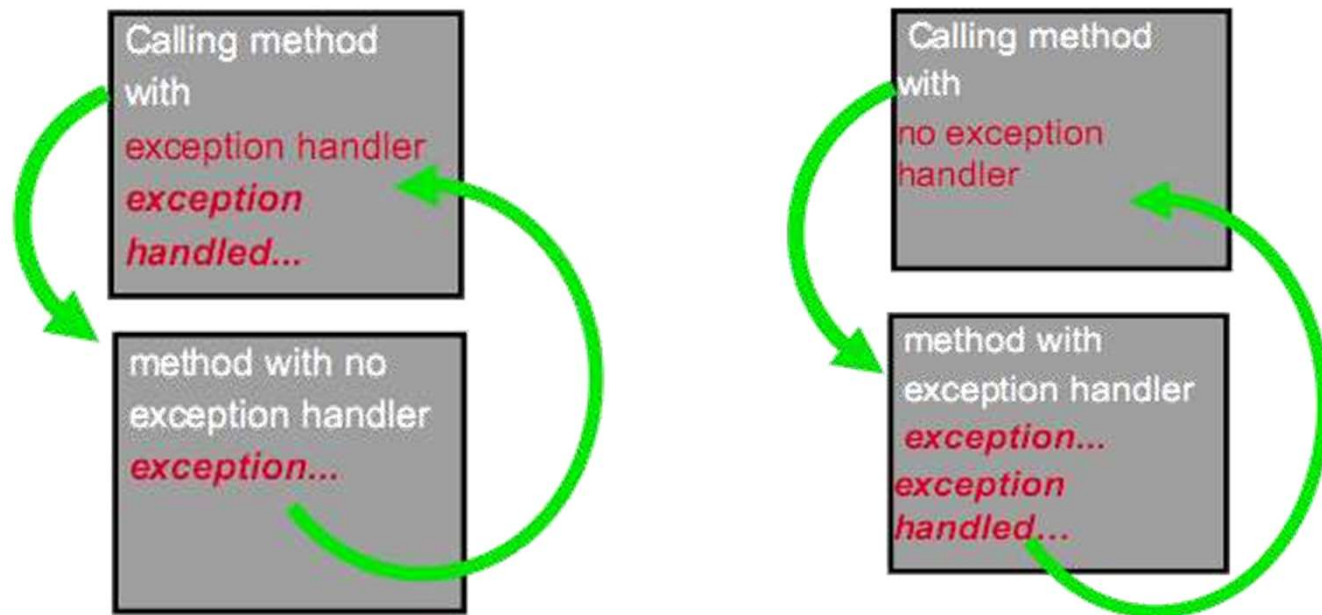
- Khi xảy ra ngoại lệ & kh xử lý =>
 - Chương trình ngắt bất thường
 - Các tài nguyên có thể không được giải phóng
- VD
 - Thao tác file I/O: nếu xảy ra ngoại lệ và kh xử lý =>
 - tài nguyên cấp phát kh đc giải phóng
 - file có thể kh truy cập được

Mô hình xử lý ngoại lệ

- Trong hướng đối tượng:
 - Đóng gói các điều kiện không mong đợi/ngoại lệ trong đối tượng
 - Khi xảy ra ngoại lệ:
 - Đối tượng tương ứng với ngoại lệ được tạo ra - chứa thông tin về ngoại lệ
 - Cung cấp cơ chế hiệu quả xử lý lỗi - tách biệt luồng điều khiển bình thường với luồng điều khiển bất thường (khi xảy ra ngoại lệ)

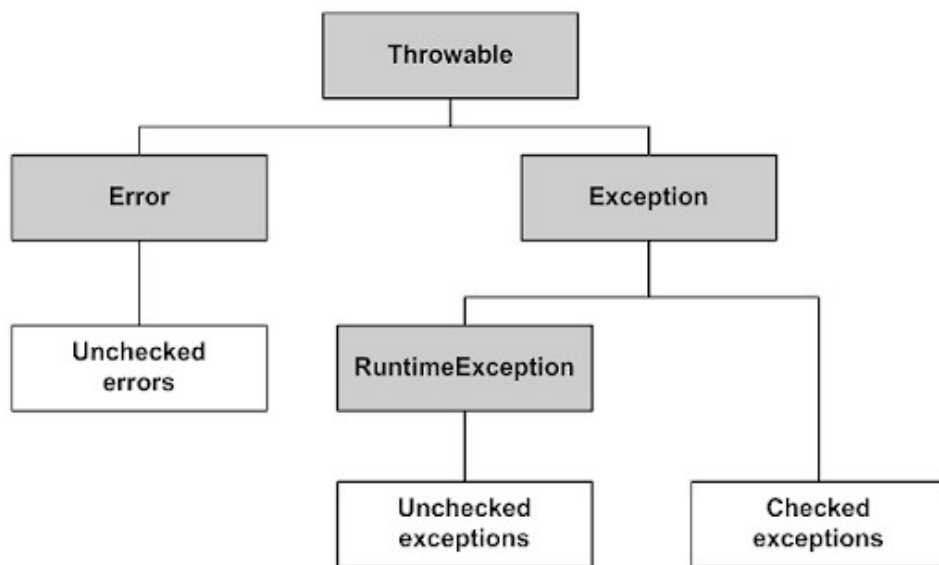
Mô hình xử lý ngoại lệ

- Ngoại lệ cần phải được xử lý ở tại phương thức sinh ra ngoại lệ hoặc ủy nhiệm cho phương thức gọi nó

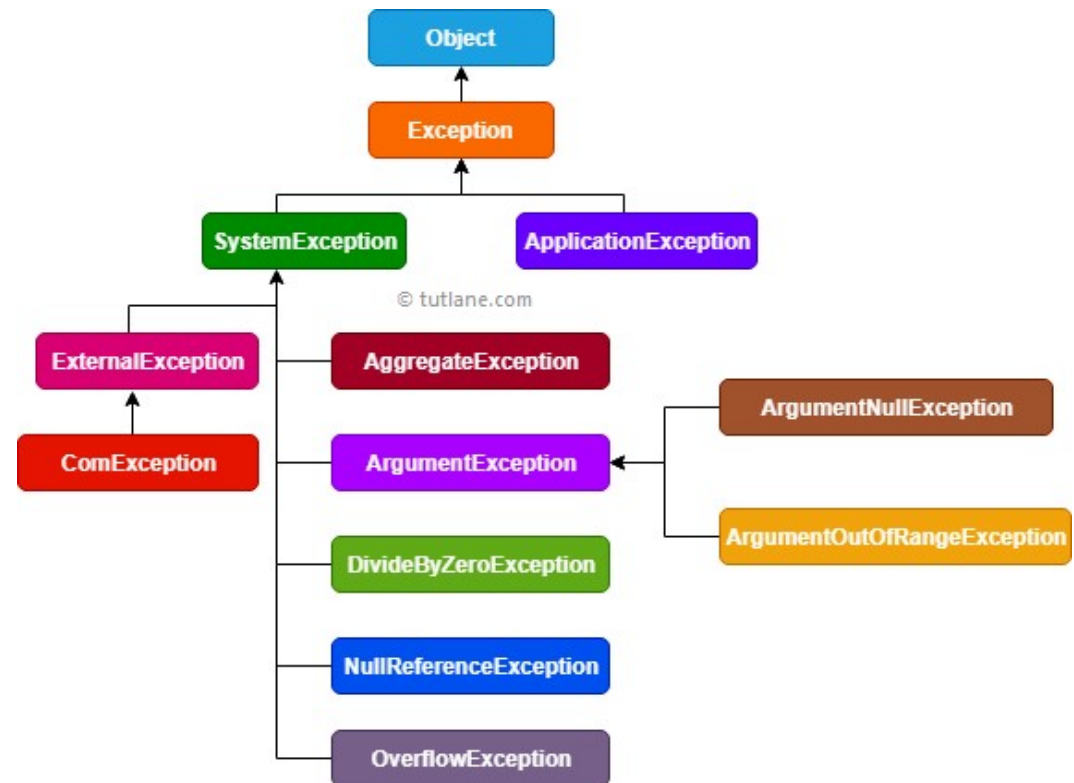


Cây phân cấp ngoại lệ trong Java, C#

■ Java



■ C#



Xử lý ngoại lệ: C#

- Xử lý theo hướng đối tượng
 - Ngoại lệ: đóng gói trong `system.exception` hoặc lớp dẫn xuất
 - Đối tượng ngoại lệ: các thông tin về ngoại lệ
- Các từ khóa
 - `try`
 - `catch`
 - `finally`

Xử lý ngoại lệ: C#

khối lệnh try ... catch

- Khối try ... catch: Phân tách đoạn chương trình thông thường và phần xử lý ngoại lệ
 - Đối tượng ex (lớp System.exception) - thông tin ngoại lệ

```
try {  
    // Khối lệnh thực thi  
}  
catch (Exception ex)  
{  
    // Khối này thực thi khi có lỗi  
    // Đối tượng Exception ex - thông tin lỗi  
}
```

- Các thông tin: ex.Source
- ex.Message

Xử lý ngoại lệ: C#

Khối try lồng nhau

- Khối try lồng nhau
 - Những phần nhỏ trong khối mã sinh ra một lỗi, nhưng toàn bộ cả khối thì lại sinh ra một lỗi khác =>

Cần có các xử lý ngoại lệ lồng nhau

```
try {  
    // khối này được giám sát để bắt lỗi  
    int[] mynumbers = new int[] {1,2,3};  
    int i = mynumbers[10];  
    try{  
        // khối này có thể phát sinh lỗi khác  
        string contents = File.ReadAllText("input.txt");  
    }  
    catch(IOException ex){  
        Console.WriteLine(ex.Message);  
    }  
    Console.WriteLine(i);  
}  
catch (Exception ex1)  
{  
    Console.WriteLine(ex1.Message);  
}
```

Xử lý ngoại lệ: C#

Nhiều khối catch

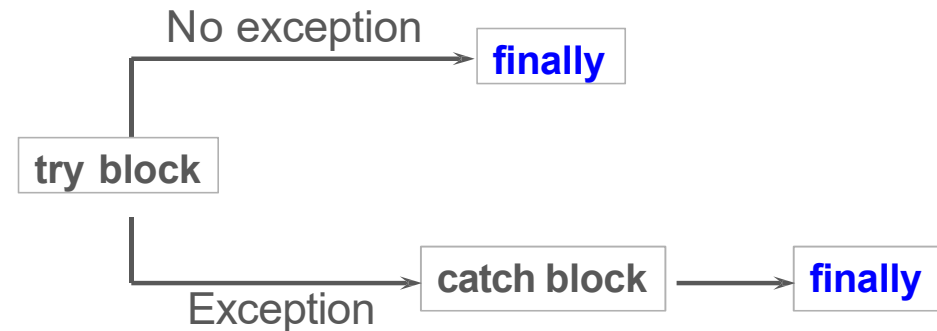
- Một đoạn mã có thể gây ra nhiều hơn một ngoại lệ => Sử dụng nhiều khối catch

```
try {  
    int x = 10;  
    int y = 0;  
    int z = x / y;  
  
}  
catch (DivideByZeroException e1) {  
    Console.WriteLine(e1.Message);  
}  
catch (Exception e2) {  
    Console.WriteLine(e2.Message);  
}
```

Xử lý ngoại lệ: C#

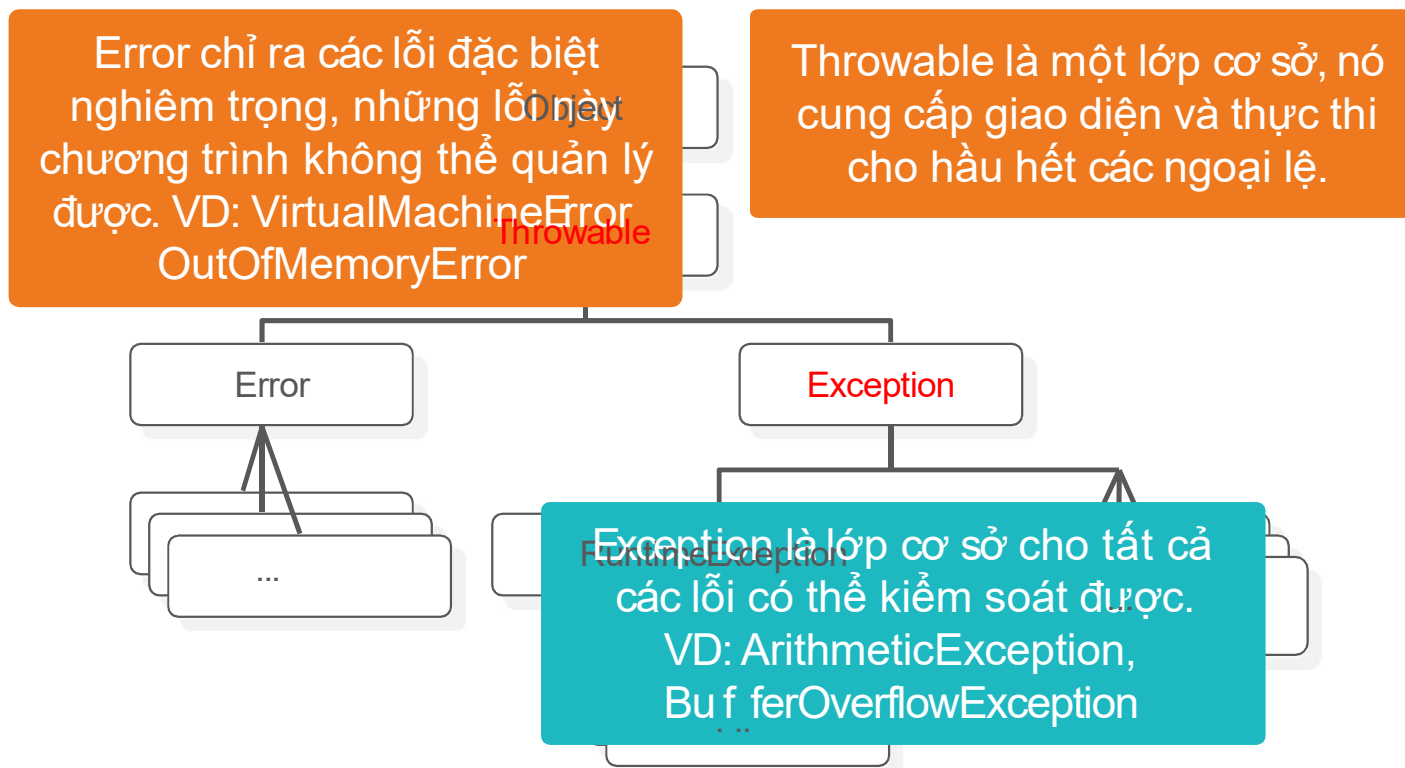
Khối finally

- Đảm bảo thực hiện tất cả các công việc cần thiết khi có ngoại lệ xảy ra, vd:
 - Đóng file, đóng socket, connection
 - Giải phóng tài nguyên (nếu cần)...
- Chắc chắn sẽ thực hiện dù ngoại lệ có xảy ra hay không



```
int x = 10; int y = 0; int z = 0;
try {
    z = x / y;
}
catch (DivideByZeroException e1) {
    Console.WriteLine(e1.Message);
}
finally {
    //Luôn được thi hành dù có phát sinh ngoại lệ hay không
    Console.WriteLine(z);
}
```

Cây phân cấp ngoại lệ trong Java

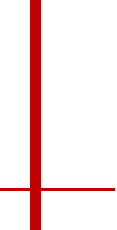


Xử lý ngoại lệ trong Java

- Xử lý ngoại lệ trong Java được thực hiện theo mô hình hướng đối tượng
 - Tất cả các ngoại lệ đều là thể hiện của một lớp kế thừa từ lớp Throwable hoặc các lớp con của nó
 - Các đối tượng này có nhiệm vụ chuyển thông tin về ngoại lệ (loại và trạng thái của chương trình) từ vị trí xảy ra ngoại lệ đến nơi quản lý/xử lý nó

Xử lý ngoại lệ trong Java

- Từ khóa liên quan / Keyword
 - try
 - catch
 - finally
 - throw
 - throws

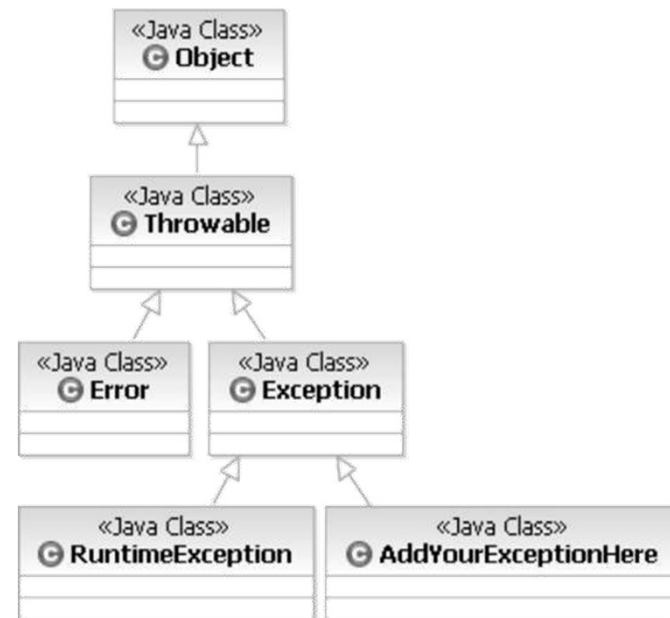


Lớp Error

- Gồm các ngoại lệ nghiêm trọng không thể kiểm tra (unchecked exception) vì có thể xảy ra ở nhiều phần của chương trình.
- Còn gọi là ngoại lệ không thể phục hồi (unrecoverable exception)
- Không cần kiểm tra trong mã nguồn Java của bạn
- Các lớp con:
 - VirtualMachineError: InternalError, OutOfMemoryError, StackOverflowError, UnknownError
 - ThreadDeath
 - LinkageError

Lớp Exception

- Chứa các loại ngoại lệ nên/phải bắt và xử lý hoặc ủy nhiệm.
- Người dùng có thể tạo ra các ngoại lệ của riêng mình bằng cách kế thừa từ Exception
- RuntimeException có thể được “tung” ra trong quá trình JVM thực hiện
 - Không bắt buộc phải bắt ngoại lệ dù có thể xảy ra lỗi
 - Không nên viết ngoại lệ của riêng mình kế thừa từ lớp này



Một số lớp con của Exception

- ClassNotFoundException, SQLException
- java.io.IOException:
 - FileNotFoundException, EOFException...
- RuntimeException:
 - NullPointerException, BufferOverflowException
 - ClassCastException, ArithmeticException
 - IndexOutOfBoundsException:
 - ArrayIndexOutOfBoundsException,
 - StringIndexOutOfBoundsException...
 - IllegalArgumentException:
 - NumberFormatException, InvalidParameterException



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

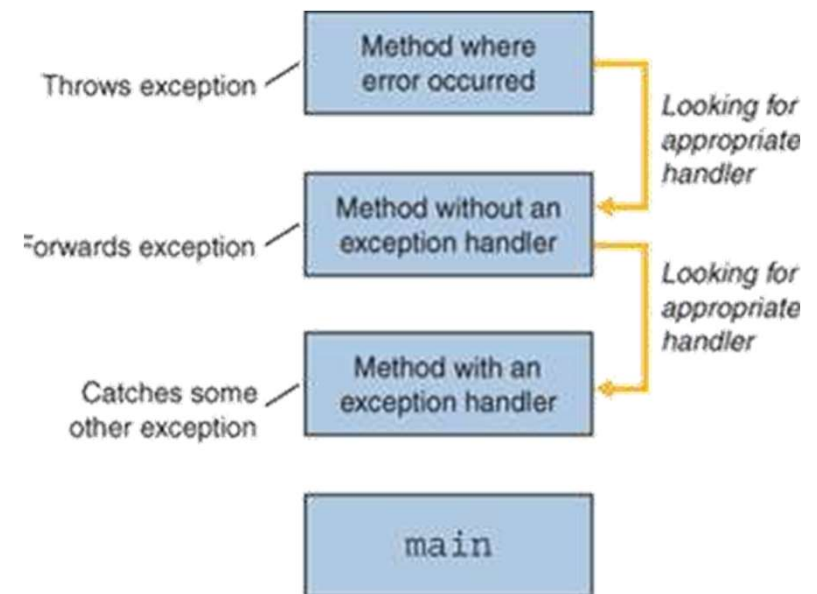
KHOA TOÁN – TIN

3. Ủy nhiệm ngoại lệ (throw)



Hai cách làm việc với ngoại lệ

- Xử lý ngay
 - Sử dụng khối try ... catch (finally nếu cần).
- Ủy nhiệm cho đoạn mã gọi nó (caller)
 - Phát sinh ngoại lệ ra ngoài phạm vi
 - Sử dụng từ khóa throw



Ủy nhiệm ngoại lệ

- Phương thức có thể ủy nhiệm ngoại lệ cho vị trí gọi nó bằng cách gọi throw
 - Tạo đối tượng exception hoặc dẫn xuất từ exception
 - phát sinh bằng throw

```
public double Thuong(double x, double y) {  
    if (y == 0) {  
        // Khởi tạo ngoại lệ  
        Exception myexception = new Exception("Số chia không được bằng 0");  
        // Phát sinh ngoại lệ  
        throw myexception;  
    }  
    return x / y;  
}
```

Ủy nhiệm ngoại lệ

- Một phương thức có thể ủy nhiệm nhiều hơn 1 ngoại lệ

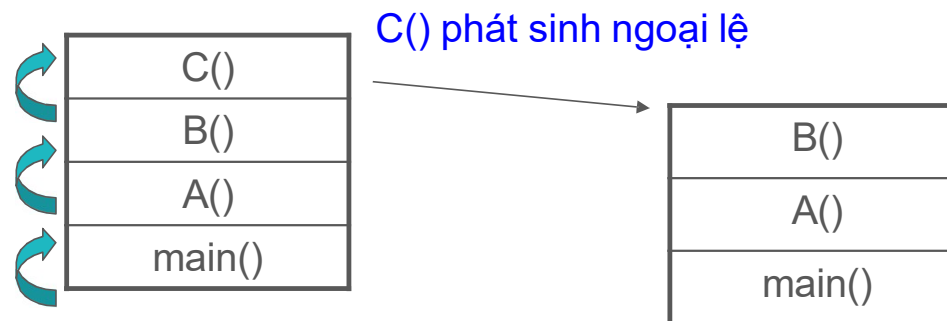
```
public void myMethod(int tuoi, String ten) {  
    if (tuoi < 18) {  
        throw new ArithmeticException("Chua du tuoi!");  
    }  
    if (ten == null) {  
        throw new Exception("Thieu ten!");  
    }  
    // ...  
}
```

Lan truyền ngoại lệ

- Phương thức gọi phương thức khác
 - Hàm main - gọi A()
 - Trong A() gọi B()
 - Trong B() gọi C()
- C() xảy ra ngoại lệ => lan truyền

Lan truyền ngoại lệ

- Nếu C() gặp lỗi và phát sinh ngoại lệ nhưng trong C() lại không xử lý ngoại lệ này => thì nơi có thể xử lý chính là nơi mà C() được gọi - B().
 - Nếu trong B() cũng không xử lý => thì phải xử lý ngoại lệ này trong A()
- => Quá trình này gọi là lan truyền ngoại lệ
- Nếu đến main() cũng không xử lý ngoại lệ được phát sinh từ C() thì chương trình sẽ phải dừng lại



Ưu điểm của ủy nhiệm ngoại lệ

- Dễ sử dụng
 - Dễ dàng chuyển điều khiển đến nơi có khả năng xử lý ngoại lệ
 - Có thể phát sinh nhiều ngoại lệ
- Tách xử lý ngoại lệ khỏi đoạn mã thông thường
- Không bỏ sót ngoại lệ (phát sinh tự động)
- Gom nhóm và phân loại các ngoại lệ
- Làm chương trình dễ đọc và an toàn hơn



ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

KHOA TOÁN – TIN

4. Ngoại lệ tự định nghĩa - user defined exception



Tạo ngoại lệ tự định nghĩa

- Các ngoại lệ do hệ thống xây dựng không đủ để kiểm soát/thể hiện đặc trưng tất cả các lỗi => Người dùng cần có thể định nghĩa các ngoại lệ - kiểm soát/gỡ lỗi tốt hơn
- Kế thừa từ exception/lớp con của nó

```
public class DataTooLongException : Exception
{
    const string errorMessage = "Dữ liệu quá dài";
    public DataTooLongException() : base(errorMessage) {
    }
}
```

Tạo ngoại lệ tự định nghĩa

```
public static void UserInput(string s) {  
    if (s.Length > 10){  
        Exception e = new DataTooLongExeption();  
        throw e;  
    }  
    //Other code  
}  
static void Main(string[] args)a  
{  
    try {  
        UserInput("Đây là một chuỗi quá dài ...");  
    }  
    catch (DataTooLongExeption e) {  
        Console.WriteLine(e.Message);  
    }  
    catch (Exception otherExeption) {  
        Console.WriteLine(otherExeption.Message);  
    }  
}
```

Tổng kết

- Ngoại lệ
 - Các sự kiện bất thường xảy ra khi chạy chương trình
- Bắt và xử lý ngoại lệ
 - Ngoại lệ được phát sinh như các đối tượng Exception, có thể được bắt và xử lý trong các khối try - catch
- Ủy nhiệm ngoại lệ
 - Định nghĩa các trường hợp đặc biệt là các ngoại lệ để có thể bắt và xử lý